FUNZIONALITA IMPLEMENTATE

Il gioco supporta partite da 2 a 5 giocatori. Il server gestisce login, lobby, scelta del colore e ready check, poi conduce la partita seguendo ere, round, ordine di turno, offer tile e piazzamento dei totem. Ogni azione viene validata lato server prima di modificare lo stato. Sono implementate carte personaggio, edificio ed evento, effetti permanenti o temporanei, costo in cibo degli edifici, bonus, sustenance, fine round e punteggio finale con gestione dei pareggi. Il client puo partire in TUI o GUI JavaFX e comunica con lo stesso server tramite Socket o RMI, anche nella stessa lobby. Le funzionalita aggiuntive coprono disconnessione e riconnessione, sospensione quando resta un solo giocatore online, timeout, salvataggio su disco e leaderboard JDBC opzionale.

DIVISIONE DEL LAVORO

- Benedetto Oliva: model, carte, effetti, regole di dominio e test del model.
- Giorgio Ortona: Socket/RMI, client controller, riconnessioni/disconnessioni, GUI e integrazione.
- Alessandro Mazzeo: controller, stati di gioco, flussi di partita e regressioni.
- Camillo Nicoletti: TUI, comandi utente, persistenza, leaderboard e deliverable.

SCELTE ARCHITETTURALI E PATTERN

Il progetto usa MVC con server autoritativo: il client invia intenzioni, mentre il server valida le azioni e conserva lo stato ufficiale. Il client mantiene solo una copia locale aggiornata dai messaggi ricevuti.

Pattern	Dove	Perche lo usiamo
State	LobbyState, TotemPlacementState, ActionResolutionState, SuspendedState, EndGameState	Incapsula transizioni e invarianti di fase; evita branch centralizzati e riduce stati illegali.
Command	GameAction e sottoclassi	Incapsula l'input utente in oggetti validabili prima dell'applicazione al dominio.
DTO / Protocol messages	serverbound, clientbound	Definisce un boundary stabile di serializzazione; rete e dominio restano disaccoppiati.
Factory + Strategy	ClientConnectionFactory, ClientSideConnection	Applica dependency inversion: il controller dipende da un'interfaccia e il trasporto si seleziona a runtime.
Adapter	Socket*Connection, RMI*Connection, VirtualView	Normalizza API diverse, stream/RMI/callback, in un contratto comune isolando i dettagli infrastrutturali.
Dispatcher	MessageDispatcher, mappe handler nel client	Usa una tabella di handler per tipo messaggio, eliminando catene instanceof e centralizzando il routing.
Observer	GameListener, callback setOnMessage, setOnDisconnect	Realizza un publish/subscribe leggero: eventi verso view e rete senza dipendere dai subscriber concreti.
Visitor	carte ed effetti	Usa double dispatch sugli effetti: comportamento specializzato senza switch sul tipo e con estensione localizzata.
Snapshot	GameStartedMessage, GameSyncMessage	Trasferisce uno stato consistente e serializzabile, utile per bootstrap e recovery idempotente.
Supervisor / Retry	ClientConnectionSupervisor	Isola la politica di reconnect con retry temporizzato, senza bloccare UI o duplicare logica di connessione.

PROTOCOLLO E PERSISTENZA

Il protocollo usa messaggi DTO serializzabili: sono oggetti semplici che trasportano dati tra client e server senza contenere logica di gioco. I ClientMessage includono LoginMessage, ReadyMessage, GameActionMessage, ReconnectMessage, DisconnectMessage, QuitMessage; i ServerMessage includono LoginAckMessage, GameStartedMessage, GameSyncMessage, StateChangedMessage, PlayerDisconnectedMessage, GameSuspendedMessage, GameResumedMessage.

Durante il gioco normale il server invia messaggi delta: solo cio che cambia. Il GameSnapshot completo viene costruito solo quando serve una sincronizzazione completa: inizio partita o riconnessione. La persistenza non salva lo snapshot, ma serializza il GameController in mesos-savestate.bin; al restart il server ricarica il controller e genera un nuovo snapshot per ogni client che si riconnette.

MESSAGGI PRINCIPALI

Direzione	Messaggi
Client -> Server	LoginMessage, ReadyMessage, GameActionMessage, ReconnectMessage, DisconnectMessage, QuitMessage
Server -> Client	LoginAckMessage, GameStartedMessage, GameSyncMessage, StateChangedMessage, PlayerDisconnectedMessage, GameSuspendedMessage, GameResumedMessage

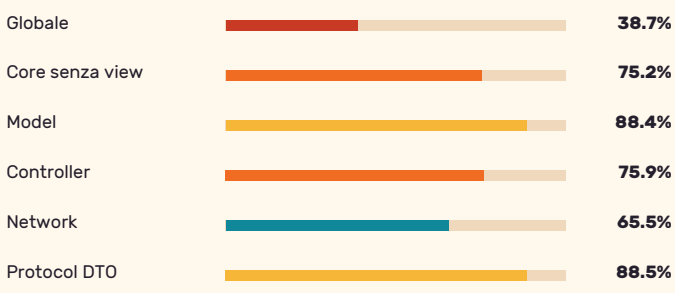
FLUSSO RECONNECT

1. **Guasto rilevato** — heartbeat, eccezione o onDisconnect
2. **Cleanup server** — LobbyManager pulisce la connessione
3. **Stato partita** — disconnessione e sospensione
4. **Retry client** — nuova richiesta ogni 2 sec
5. **Riallineamento** — ack, snapshot e stato corrente

TEST E COPERTURA

La suite contiene 95 classi di test su model, controller, stati, network, protocollo, persistenza e riconnessioni. La copertura e misurata con JaCoCo tramite mvn clean verify. GUI/TUI, launcher e JDBC leaderboard sono validati anche con prove manuali perche dipendono da interazione grafica, processo o database esterno.

Copertura istruzioni JaCoCo



Indicatori sintetici

Test class	95
Branch globale	36.0%
Linee globali	41.6%
Core linee	73.5%
Comando coverage	mvn clean verify
Validazione manuale	GUI/TUI, launcher, JDBC